Sparse Similarity-Aware Label Smoothing via Learned Latent Spaces

Nihar Kalode
Department of Computer Science
University of Illinois Urbana-Champaign
nkalode2@illinois.edu

Abstract

Label smoothing is widely used to reduce neural network overconfidence, but its standard uniform form ignores label-space structure by distributing probability mass across all non-target classes. This produces dense, diffuse targets in which semantically plausible alternatives and unrelated labels receive the same treatment. We argue that effective label smoothing depends not only on the smoothing strength but also on the support over which smoothing mass is assigned: smoothing should be sparse and similarity-aware. We instantiate this principle with Sparse Similarity-Aware Label Smoothing (SSA-LS), a simple and practical method that uses latent-space class neighborhoods to redistribute probability mass only toward plausible alternatives. Across multiple datasets and architectures, SSA-LS consistently improves confidence calibration while preserving, and sometimes improving, accuracy. We further show that strong representations do not inherently guarantee well-calibrated predictions; calibration improves when smoothing explicitly exploits their semantic structure. Sparsity ablations, neighborhood corruption studies, and latent-space comparisons demonstrate that the gains arise from structured, similarity-aware sparsity, establishing it as a critical ingredient for label smoothing.

1 Introduction

Misclassifying a truck as a cat is clearly wrong. So is calling it a car. But are these mistakes equally severe? This distinction matters not only for accuracy, but also for confidence: a model that assigns uncertainty to plausible alternatives is qualitatively different from one that spreads or concentrates probability mass without regard to label-space structure. Modern deep neural networks achieve high top-1 accuracy, but their predicted probabilities are often poorly aligned with empirical correctness [4]. This miscalibration gap provides unreliable confidence estimates, and can contaminate downstream decision-making, which is an especially significant problem for risk-aware systems that depend on well-calibrated confidence scores. This has motivated extensive work on calibration metrics and procedures, such as reliability diagrams and expected calibration error (ECE), as well as post-hoc methods such as temperature scaling [2, 17, 16, 4].

Perhaps the most widely adopted training-time remedy is *label smoothing*, which replaces one-hot targets with a convex combination of the hard label and a prior distribution over classes, typically uniform [21]. By discouraging sharply peaked predictive distributions, label smoothing acts as an effective regularizer to mitigate overconfident predictions [15, 19]. More broadly, training strategies that mix or soften labels—such as mixup—attribute part of their calibration benefit to the implicit label smoothing effect [22].

However, uniform label smoothing [21] makes a strong modeling assumption: all misclassifications are equally severe. For many recognition problems, however, classes are not equally confusable. A

Preprint.

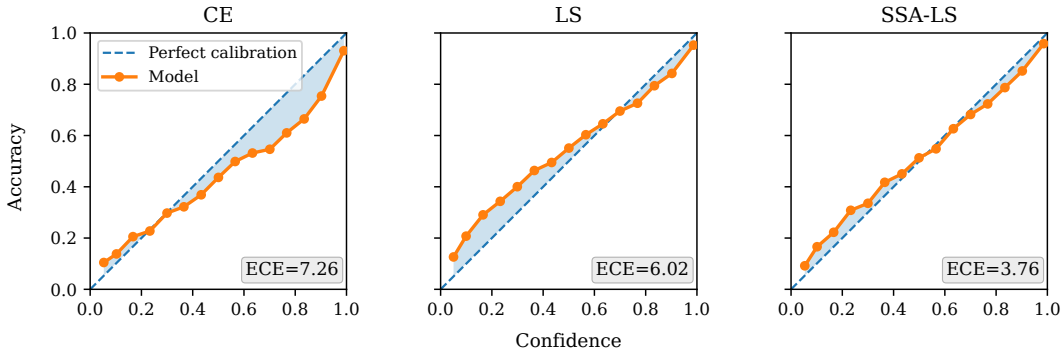


Figure 1: Reliability diagrams [2, 17] comparing ResNet-18 models trained on TinyImageNet using Cross Entropy (CE), Uniform Label Smoothing (LS) [21], and SSA-LS (ours) targets. The dashed diagonal denotes perfect calibration, and the orange curve shows empirical accuracy within confidence bins. SSA-LS moves the reliability curve closer to the diagonal and substantially reduces ECE compared with CE and LS, illustrating that restricting smoothing mass to semantically plausible alternatives improves confidence calibration.

model that is uncertain between semantically or visually similar categories is arguably less problematic than one that spreads uncertainty onto unrelated classes. Recent work [14, 25, 23, 13] has explored *adaptive* smoothing schemes that allocate probability mass based on class-based or instance-based similarity rather than uniformly. We ask a complementary question: must smoothing assign non-zero probability to every incorrect class at all? This idea motivates an appealing principle: label smoothing should smooth *toward plausible alternatives only*.

In this work, we study **similarity-aware sparsity** as a design principle for label smoothing. Our central claim is that calibration depends not only on the amount of smoothing, but also on the *support* over which smoothing mass is distributed. We instantiate this principle with **Sparse Similarity-Aware Label Smoothing** (SSA-LS), a simple drop-in target construction rule that assigns smoothing mass only to semantically plausible neighboring classes. Our main contributions are as follows:

- We show that **similarity-aware sparsity** is a key driver of calibration. Across sparsity levels, calibration changes substantially while accuracy remains largely stable, indicating that the smoothing support matters independently of how much mass is assigned.
- We introduce SSA-LS, which requires only a one-time precomputation per dataset and incurs *no training-time overhead*. Across architectures and datasets, SSA-LS achieves best calibration among label smoothing methods, outperforming standard label smoothing and recent adaptive variants such as OLS [25] and ILS [14].
- We show that sparsity must be *structured*: random or corrupted neighborhoods degrade calibration, while neighborhoods derived from different latent spaces, including pretrained CLIP/DINO representations and an in-domain β-VAE, yield similar gains. We also show that strong representations alone do not guarantee calibrated predictions.

2 Preliminaries

Let $\mathcal{X}$ and $\mathcal{Y} = \{1, 2, \ldots, C\}$ denote the input and output spaces respectively, where C is the number of classes present. We train neural network $h_\theta : \mathcal{X} \to \mathbb{R}^C$ that outputs logits $\boldsymbol{z} = h_\theta(\boldsymbol{x})$ for input $\boldsymbol{x} \in \mathcal{X}$ to predict the true class label $y \in \mathcal{Y}$. Here, the model then predicts class $\hat{y} = \arg\max_c p_\theta(c|\boldsymbol{x})$ with associated confidence $\hat{p} = \max_c p_\theta(c|\boldsymbol{x})$ where $p_\theta(c|\boldsymbol{x}) = \mathrm{softmax}_c(h_\theta(\boldsymbol{x}))$. A classifier is perfectly calibrated if

$$\mathbb{P}(\hat{y} = y|\hat{p} = p) = p, \quad \forall p \in [0, 1] \tag{1}$$

Essentially, it means that among all predictions made with confidence p, the model is correct $100p\%$ times.

Label Smoothing. Label smoothing replaces the one-hot target distribution with a softened target. Let the smoothing strength ϵ denote the probability mass redistributed among incorrect classes, which may depend on the sample $\boldsymbol{x}$, true label y, and timestep t for adaptive or online label smoothing methods (e.g. IVON [23], OLS [25]). In the general case, we have target distribution

$$q(c \mid \boldsymbol{x}, y, t) = (1 - \epsilon(\boldsymbol{x}, y, t))\mathbb{1}\{c = y\} + \epsilon(\boldsymbol{x}, y, t) \cdot s(c \mid \boldsymbol{x}, y, t) \tag{2}$$

where $s(c \mid \boldsymbol{x}, y, t)$ is a valid probability distribution such that $s(y|\boldsymbol{x}, y, t) = 0$.

Uniform label smoothing (LS) [21] is the standard and simple method where a fixed ϵ is redistributed uniformly over all incorrect classes. Specifically, we have

$$q_\epsilon^{\mathrm{LS}}(c \mid y) = \begin{cases} 1 - \epsilon, & \text{if } c = y, \\ \dfrac{\epsilon}{C - 1}, & \text{otherwise.} \end{cases} \tag{3}$$

Metrics. Expected calibration error, a useful scalar summary statistic of calibration [4], is the expectation of difference between the model's confidence and accuracy

$$\mathbb{E}_{\hat{p}}\left[|\mathbb{P}\left(\hat{y} = y|\hat{p} = p\right) - p|\right] \tag{4}$$

To approximately compute ECE, we partition predictions into M equal intervals [16, 4]. Let interval $I_m = \left(\frac{m-1}{M}, \frac{m}{M}\right]$ and B_m be the corresponding set of indices of samples based on prediction confidences. Then, we define

$$\mathrm{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{1}(\hat{y}_i = y_i) \tag{5}$$

and

$$\mathrm{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i \tag{6}$$

Finally, we may approximate ECE with the interval bins as the weighted average

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} |\mathrm{acc}(B_m) - \mathrm{conf}(B_m)| \tag{7}$$

where n is the total number of samples and, in practice, M is set to 15. Observe that for a perfectly calibrated model we have $\mathrm{acc}(B_m) = \mathrm{conf}(B_m)$ for all m i.e. $\mathrm{ECE} = 0$. This is the primary metric we employ to compare performance of label smoothing methods ahead.

We also consider classification accuracy, negative log likelihood (NLL), and maximum calibration error [16] defined as

$$\mathrm{MCE} = \max_m |\mathrm{acc}(B_m) - \mathrm{conf}(B_m)| \tag{8}$$

as additional metrics. ECE and MCE are reported as percentages (%), as is classification accuracy. For visual aid, we rely on reliability diagrams [2, 17].

3 Similarity-Aware Sparsity for Label Smoothing

Overview. We propose a general recipe to incorporate structured sparsity, based on a learned latent space, into training target distributions. A fixed latent encoder is used to extract features for all train samples, from which class-wise centroids and inter-class distances are computed. For each class, these distances are used to identify a fixed number of plausible misclassifications, and the smoothing mass is redistributed among them in a similarity-aware manner. This procedure yields a sparse, similarity-aware, and smooth target distribution for training. We further introduce SSA-LS, a simple instantiation of this framework that enables controlled analysis of similarity-aware sparsity.

Construction. Let f denote the chosen encoder, and $\boldsymbol{z}_i \in \mathbb{R}^d$ such that $\boldsymbol{z}_i = f(\boldsymbol{x}_i)$ denote the latent representation of sample $\boldsymbol{x}_i$ where d is the corresponding latent space's dimensionality. Let $D_i = \{\boldsymbol{x}_n : y_n = i\}$. Then, for class D_i, we have the latent class centroid

$$\boldsymbol{\mu}_i = \frac{1}{|D_i|} \sum_{\boldsymbol{x} \in D_i} \boldsymbol{z}_i \tag{9}$$

The class-wise distances are computed based on these centroids, where the metric $d^{(e)}$ is chosen to match the inductive bias of the representation space. In practice, we use cosine distance d_c for CLIP [20] and DINO [18], and euclidean distance d_e for β-VAE [6].

$$d_c(\boldsymbol{\mu}_i, \boldsymbol{\mu}_j) = 1 - \frac{\boldsymbol{\mu}_i^\top \boldsymbol{\mu}_j}{\|\boldsymbol{\mu}_i\| \|\boldsymbol{\mu}_j\|}, \tag{10}$$

$$d_e(\boldsymbol{\mu}_i, \boldsymbol{\mu}_j) = \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|_2 \tag{11}$$

The framework itself is agnostic to the choice of latent representation.

Now, we introduce the sparsity factor k. Let $\mathcal{N}_k(i)$ denote the set containing the k closest neighbors of the ith class under the chosen distance metric $d^{(e)}$. Let $q(c \mid \boldsymbol{x}, y, t)$ denote the target distribution produced by a base label smoothing method as defined in (2). We then construct a sparse similarity-aware variant $q^{\text{SSA}}(c \mid \boldsymbol{x}, y, t)$, by redistributing the non-target probability mass among classes in $\mathcal{N}_k(y)$:

$$q^{\text{SSA}}(c \mid \boldsymbol{x}, y, t) = (1 - \epsilon(\boldsymbol{x}, y, t))\mathbb{1}\{c = y\} + \epsilon(\boldsymbol{x}, y, t) \cdot s^{\text{SSA}}(c \mid \boldsymbol{x}, y, t) \tag{12}$$

where the sparse similarity-aware redistribution is

$$s^{\text{SSA}}(c \mid \boldsymbol{x}, y, t) = \frac{s(c \mid \boldsymbol{x}, y, t) \cdot \mathbb{1}\{c \in \mathcal{N}_k(y)\}}{\sum_{j \in \mathcal{N}_k(y)} s(j \mid \boldsymbol{x}, y, t)} \tag{13}$$

To study the effect of similarity-aware sparsity, we introduce SSA-LS, a simple label smoothing method based on the precomputed class-wise distances. We define

$$q_\epsilon^{\text{SSA-LS}}(c \mid y) = \begin{cases} 1 - \epsilon, & \text{if } c = y, \\ \epsilon \cdot \dfrac{\exp\big(-d^{(e)}(\boldsymbol{\mu}_c, \boldsymbol{\mu}_y)/T\big)}{\sum\limits_{j \in \mathcal{N}_k(y)} \exp\big(-d^{(e)}(\boldsymbol{\mu}_j, \boldsymbol{\mu}_y)/T\big)}, & \text{if } c \in \mathcal{N}_k(y), \\ 0, & \text{otherwise.} \end{cases} \tag{14}$$

where $T > 0$ is a temperature parameter set to $T = 1.2$. We deliberately adopt a simple class-wise scheme to isolate the effect of sparsity under a fixed similarity structure.

4 Experiments & analysis

4.1 Setup

Datasets. The CIFAR-100 [11] dataset contains 32×32 color images from 100 classes. It contains a total of 50,000 train and 10,000 test images. The TinyImageNet dataset, a subset of ImageNet [3], contains 64×64 color images from 200 classes, with 100,000 training images and 10,000 validation images. For data augmentation, we apply random cropping and horizontal flipping [12] to the training sets. Apart from dataset normalization, no other preprocessing is performed.

Architectures. For image classification, we evaluate 5 architectures: DenseNet-121 [8], MobileNetV3-Large [7], ResNet-18, ResNet-34 [5], and WideResNet-28-10 [24]. The latent spaces considered to construct similarity-aware targets are CLIP ViT-B/32 (OpenAI pretrained) [20], DINOv2 ViT-B/14 [18], and β-VAE [10, 6]. CLIP and DINOv2 are used as fixed pretrained encoders, while the classification models and β-VAE are trained from scratch. Unless specified otherwise, CLIP is used to construct SSA-LS targets.

Table 1: Calibration and accuracy comparison across architectures on CIFAR-100. We report results using CE, LS [21], OLS [25], ILS [14], and SSA-LS (ours) targets. SSA-LS uses a sparsity level k selected from preliminary sparsity sweeps and fixed across all seeds for each architecture; selected values are reported in Appendix A. Across all architectures, SSA-LS consistently achieves the lowest ECE, while MCE and NLL are also improved in most cases. Accuracy remains comparable to, or even exceeds, baseline methods. Additional results are reported in Appendix B.

Model	Method	ECE ↓	Accuracy ↑	NLL ↓	MCE ↓
DenseNet-121	CE	10.26 ± 0.33	$\mathbf{67.99 \pm 0.41}$	1.32 ± 0.01	21.29 ± 0.94
	LS	5.89 ± 0.53	67.64 ± 0.16	1.35 ± 0.01	15.66 ± 1.92
	OLS	17.04 ± 0.35	66.93 ± 0.11	1.48 ± 0.02	38.48 ± 1.48
	ILS	9.64 ± 0.39	67.81 ± 0.33	1.33 ± 0.01	24.84 ± 10.51
	SSA-LS	$\mathbf{4.58 \pm 0.51}$	67.68 ± 0.29	$\mathbf{1.28 \pm 0.01}$	$\mathbf{11.32 \pm 1.88}$
ResNet-18	CE	4.49 ± 0.16	78.45 ± 0.23	0.87 ± 0.01	14.58 ± 5.41
	LS	5.84 ± 0.31	79.13 ± 0.10	0.93 ± 0.00	20.60 ± 1.56
	OLS	3.43 ± 0.17	$\mathbf{79.32 \pm 0.20}$	$\mathbf{0.81 \pm 0.00}$	12.88 ± 4.74
	ILS	4.31 ± 0.18	78.64 ± 0.30	0.87 ± 0.01	15.01 ± 2.83
	SSA-LS	$\mathbf{2.34 \pm 0.35}$	78.62 ± 0.21	0.84 ± 0.01	$\mathbf{12.82 \pm 1.55}$
WRN-28-10	CE	4.64 ± 0.17	81.04 ± 0.18	0.78 ± 0.01	17.46 ± 6.51
	LS	3.97 ± 0.48	81.07 ± 0.18	0.82 ± 0.01	15.86 ± 6.68
	OLS	8.74 ± 0.19	80.71 ± 0.23	0.84 ± 0.00	39.53 ± 30.51
	ILS	4.52 ± 0.34	$\mathbf{81.29 \pm 0.23}$	0.78 ± 0.01	13.16 ± 2.75
	SSA-LS	$\mathbf{3.20 \pm 0.14}$	80.97 ± 0.31	$\mathbf{0.76 \pm 0.01}$	$\mathbf{12.87 \pm 1.40}$

Training and hyperparameters. All models are trained under the same optimization protocol. We use SGD with Nesterov momentum 0.9, weight decay 5×10^{-4}, and cosine learning-rate decay for 200 epochs with an initial learning rate of 0.1. For all smoothing-based methods, we fix the smoothing strength to $\epsilon = 0.02$, unless specified otherwise. This is consistent with the small smoothing strengths commonly considered in calibration-oriented label-smoothing work [15, 14]. Full training details are presented in Appendix A.

All results report mean $\pm$ standard deviation over 5 seeds.

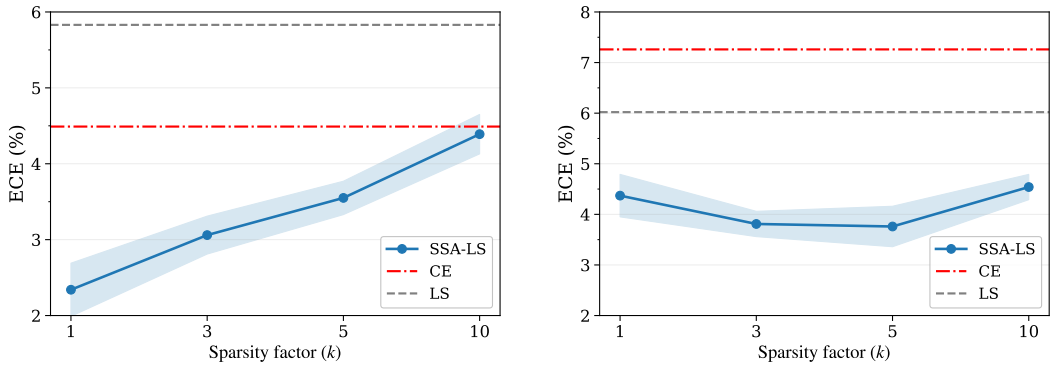
4.2 SSA-LS consistently improves calibration

We first evaluate whether sparse similarity-aware redistribution improves calibration across architectures. Following the experimental protocol in subsection 4.1, all smoothing-based methods use the same fixed smoothing strength $\epsilon = 0.02$. This fixes the total smoothing budget and isolates the effect of *where* the non-target probability mass is assigned.

Table 1 reports results on CIFAR-100 across three architectures. SSA-LS achieves the lowest ECE for every architecture, outperforming all evaluated baselines: cross entropy (CE), uniform label smoothing (LS) [21], online label smoothing (OLS) [25], and instance-based label smoothing (ILS) [14]. The consistency of these gains is important: calibration is known to depend on model-design choices such as depth and width [4], yet SSA-LS improves calibration across all evaluated backbones; additional backbones are reported in Appendix B.

Importantly, SSA-LS does not buy calibration by sacrificing predictive performance. Although regularization-based calibration methods can induce an accuracy–calibration trade-off [26], SSA-LS maintains accuracy comparable to the strongest baselines and often improves it. Thus, the gains are not explained by stronger regularization, but by a better use of the same smoothing budget: concentrating probability mass on semantically plausible alternatives rather than spreading it uniformly across all non-target classes. It also reduces MCE and NLL in most settings.

We observe the same qualitative trend beyond CIFAR-100. On TinyImageNet, SSA-LS again improves calibration across the evaluated architectures while maintaining competitive accuracy; full results are provided in Appendix B. Together, these results show that sparse similarity-aware redistribution yields consistently better-calibrated models across dataset–architecture pairs without sacrificing predictive accuracy.



(a) CIFAR-100 (b) TinyImageNet

Figure 2: Impact of sparsity on calibration (ECE ↓) across datasets. Calibration is highly sensitive to sparsity: SSA-LS achieves optimal performance at small to moderate k, while larger k degrades performance by diluting similarity-aware structure and approaching uniform label smoothing. Horizontal lines denote CE and LS baselines. Error bands indicate ± 1 standard deviation across seeds.

4.3 Effects of sparsity (k)

Since our sparse label smoothing protocol redistributes probability mass among similar classes, the sparsity factor k—the number of classes receiving this mass—is a central design choice. We show that sparsity is not a secondary hyperparameter, but a defining mechanism governing the effectiveness of label smoothing. In particular, restricting redistribution to a small, semantically meaningful neighborhood is essential for achieving strong calibration.

Redistribution smoothing mass across many classes weakens the locality assumption underlying similarity-aware smoothing methods. In large-scale settings, this effect is amplified: if mass is spread more broadly, the target distribution approaches uniform label smoothing, since variations in allocation become negligible relative to the dominant true class mass $1 - \epsilon$. As a result, the benefits of incorporating semantic similarity are diminished. Within SSA-LS, we observe this behavior concretely: increasing k consistently degrades calibration, with large-k regimes effectively collapsing to uniform smoothing.

Conversely, overly restrictive sparsity underutilizes available neighborhood information and increases sensitivity to noise in the latent structure. These observations highlight a fundamental trade-off: effective smoothing must preserve semantic locality while leveraging sufficient neighborhood information. We therefore hypothesize that *moderate sparsity*—dependent on dataset scale and underlying class structure—yields optimal calibration by preserving tight, semantically meaningful neighborhoods while maximizing the exploitable information within them. We validate this hypothesis through controlled ablations over k (Figure 2), using ResNet-18 as a controlled setting.

For CIFAR-100, we observe that the ECE increases monotonically with k, with the lowest ECE achieved at $k = 1$. In contrast, TinyImageNet exhibits a non-monotonic relationship, where ECE decreases to a minimum at moderate sparsity ($k \approx 3$–5) before increasing again as sparsity is further relaxed.

These results highlight the dataset-dependent nature of optimal sparsity. For CIFAR-100, which contains only 100 classes, redistributing the smoothing mass to strictly the closest neighbor appears sufficient to capture all meaningful semantic relationships, while additional neighbors introduce noise and dilute locality. On the other hand, TinyImageNet benefits from slightly larger neighborhoods, suggesting that richer label spaces require broader—but still constrained—support for effective smoothing. This reinforces that sparsity should be treated as a primary design choice, governed by dataset scale and structure, rather than a secondary hyperparameter. While the exact optimal k exhibits architecture dependence, the overall trend remains consistent: calibration improves at small-to-moderate sparsity and degrades as k increases.

The accuracy–calibration trade-off commonly associated with regularization methods remains a potential concern here [26]. However, we observe that classification accuracy remains largely stable across sparsity levels. Mean top-1 accuracy varies narrowly from 78.62% to 79.02% for CIFAR-100 and 65.38% to 65.84% for TinyImageNet. Although minor variations are present, they are small in magnitude and do not exhibit a consistent trade-off with calibration. Pooled accuracy over all k values remains stable at $78.94 \pm 0.31\%$ and $65.62 \pm 0.28\%$ for CIFAR-100 and TinyImageNet, respectively (see Table 8 for complete results).

This indicates that sparsity primarily affects confidence calibration rather than predictive performance. Consequently, controlling k provides a practical mechanism for reducing ECE with minimal impact on accuracy.

4.4 Strong representations do not induce calibration

Employing a learned latent space to impose sparsity in target construction implicitly assumes that the encoder captures meaningful class structure. Prior work [9, 1] has shown that even simple nearest-neighbor classifiers trained on well-structured representations achieve strong accuracy, raising a critical concern: do strong representations alone induce good calibration?

To address this, we evaluate in a frozen-feature setting. We extract latent embeddings using the pretrained CLIP ViT-B/32 [20] image encoder and train a classifier directly on these fixed features. The classifier is a lightweight 2-layer MLP defined as

$$\text{Linear}(d \to h) \to \text{ReLU} \to \text{Dropout}(p = 0.2) \to \text{Linear}(h \to C) \tag{15}$$

where d is the latent feature size, C is the number of classes, and h is set to 1024.

Importantly, the representation remains entirely fixed and only a small classifier is trained on top. This setup removes any confounding effects of representation learning or model capacity, and isolates the impact of target constructions (CE, LS, SSA-LS) on model calibration. We ablated over $\epsilon \in \{0.01, 0.02, 0.05, 0.1\}$, as reported in Table 2.

Table 2: Calibration and accuracy comparison (mean $\pm$ std) on CIFAR-100 using a lightweight MLP trained on frozen features extracted from CLIP encoder, averaged over 5 seeds. The model trained on Cross Entropy (CE) targets achieves competitive accuracy but poor calibration. A well-structured latent space does not guarantee well-calibrated models.

Method	ϵ	ECE $\downarrow$	Accuracy $\uparrow$	NLL $\downarrow$	MCE $\downarrow$
CE	–	9.03 ± 0.20	78.12 ± 0.18	0.86 ± 0.00	19.98 ± 0.74
LS	0.01	5.11 ± 0.22	78.43 ± 0.24	0.78 ± 0.00	13.01 ± 0.90
LS	0.02	2.57 ± 0.24	78.65 ± 0.26	$\mathbf{0.76 \pm 0.00}$	9.56 ± 2.40
LS	0.05	2.83 ± 0.23	78.82 ± 0.13	0.78 ± 0.00	$\mathbf{7.31 \pm 1.49}$
LS	0.10	9.47 ± 0.24	$\mathbf{79.04 \pm 0.21}$	0.84 ± 0.00	16.23 ± 0.88
SSA-LS	0.01	6.19 ± 0.13	78.22 ± 0.16	0.80 ± 0.00	17.04 ± 1.71
SSA-LS	0.02	4.05 ± 0.15	78.27 ± 0.18	0.78 ± 0.00	14.55 ± 3.20
SSA-LS	0.05	$\mathbf{2.03 \pm 0.19}$	78.17 ± 0.20	0.78 ± 0.00	12.52 ± 0.57
SSA-LS	0.10	6.77 ± 0.23	78.02 ± 0.15	0.82 ± 0.00	17.41 ± 11.34

We find that strong pretrained representations alone do not yield well-calibrated models. Despite competitive accuracy (78.12%), the CE baseline exhibits poor calibration (ECE 9.03), indicating that high-quality features do not inherently produce reliable confidence estimates. Uniform label smoothing (LS) substantially improves calibration (ECE 2.57), and SSA-LS further reduces ECE to 2.03, highlighting that how the smoothing mass is distributed between classes is critical. Importantly, even in strong representation regimes, incorporating representation-aware priors into target distributions remains essential for achieving well-calibrated predictions.

4.5 Structured sparsity: importance and robustness

Similarity-aware vs. naive sparsity. The question arises: what if we discard the learned latent space and impose random sparsity? Here, we investigate two claims: (i) sparsity must be similarity-aware, and (ii) SSA-LS is robust to imperfect representations.

We conduct controlled corruption experiments on the similarity structure used to construct SSA-LS targets. We artificially corrupt the target construction: for each class, we replace a fraction p of its top-k neighbors with randomly selected classes (not in the original top-k), progressively degrading the semantic quality of the neighborhood while keeping sparsity fixed. Setting $k = 5$, we train using these corrupted targets (Table 3).

Table 3: Effect of neighborhood corruption on calibration and accuracy (ResNet-18 on CIFAR-100). Corruption replaces a fraction of top-k neighbors with random classes. Random sparsity performs substantially worse than similarity-aware sparsity, and SSA-LS degrades toward the CE baseline only under extreme corruption.

Method	Corruption	ECE ↓	Accuracy ↑	NLL ↓	MCE ↓
CE	–	4.49 ± 0.16	78.45 ± 0.23	0.87 ± 0.01	14.58 ± 5.41
LS	–	5.84 ± 0.31	$\mathbf{79.13 \pm 0.10}$	0.93 ± 0.00	20.60 ± 1.56
SSA-LS	0.0	$\mathbf{3.49 \pm 0.27}$	78.97 ± 0.21	$\mathbf{0.83 \pm 0.01}$	15.95 ± 2.72
SSA-LS	0.2	3.79 ± 0.40	78.95 ± 0.36	0.85 ± 0.01	$\mathbf{14.42 \pm 1.58}$
SSA-LS	0.4	3.85 ± 0.31	78.98 ± 0.21	0.87 ± 0.01	17.14 ± 2.04
SSA-LS	0.6	3.98 ± 0.30	78.76 ± 0.19	0.91 ± 0.00	17.00 ± 2.12
SSA-LS	0.8	4.25 ± 0.49	78.87 ± 0.32	0.94 ± 0.01	18.68 ± 1.31
SSA-LS	1.0	4.29 ± 0.16	78.72 ± 0.20	0.97 ± 0.01	18.49 ± 1.72

SSA-LS exhibits graceful degradation: as the corruption level increases ($p = 0.0 \rightarrow 1.0$), calibration performance (ECE, MCE, NLL) degrade steadily rather than collapsing, while accuracy remains largely stable. Crucially, SSA-LS outperforms CE across a wide range of corruption levels and only approaches CE under extreme corruption, where semantic structure is entirely destroyed. This directly confirms that similarity-aware sparsity is optimal for improving calibration and that SSA-LS is robust to poor representations.

Robustness to latent space choice. Our formulation defines semantic neighborhoods using distances between class centroids in a latent space. This raises a natural question: are the gains tied to a particular representation, or does SSA-LS only require a reasonably meaningful notion of class similarity? To test this, we construct targets using three substantially different latent spaces: CLIP and DINO, which are pretrained visual representations, and a β-VAE trained directly on the dataset in an unsupervised manner.

Table 4 shows that SSA-LS is robust to the choice of latent space. Across two architectures, CLIP, DINO, and β-VAE all reduce ECE relative to CE while maintaining comparable accuracy. The measurements are also largely similar across latent spaces: while CLIP performs best overall, DINO and β-VAE remain close.

These results suggest that SSA-LS does not depend on a single privileged encoder. Pretrained representations such as CLIP and DINO provide strong off-the-shelf neighborhoods, but an in-domain β-VAE also yields consistent calibration gains. Thus, the key ingredient is not the specific latent model, but the presence of a useful class-neighborhood structure for sparsely redistributing smoothing.

5 Related Work

The miscalibration of modern neural networks has been extensively documented [17, 4]. Guo et al. [4] show that contemporary deep models are often systematically overconfident and demonstrate that temperature scaling, a simple post-hoc rescaling of logits, can substantially improve calibration. Thulasidasan et al. [22] show that data augmentation techniques such as mixup, which trains models on convex combinations of inputs and labels, can also improve calibration by encouraging smoother

Table 4: Robustness to latent space choice. Setting $k = 5$, we construct SSA neighborhoods using CLIP, DINO, and β-VAE latent spaces. Across ResNet-18 and WideResNet-28-10 on CIFAR-100, all latent spaces improve ECE over CE while maintaining comparable accuracy, indicating that SSA-LS is robust to the representation used to define class similarity.

(a) ResNet-18

Latent Space	ECE ↓	Accuracy ↑
CE	4.49 ± 0.16	78.45 ± 0.23
CLIP	3.55 ± 0.22	79.02 ± 0.23
DINO	3.69 ± 0.37	78.96 ± 0.31
β-VAE	3.65 ± 0.21	78.87 ± 0.10

(b) WideResNet-28-10

Latent Space	ECE ↓	Accuracy ↑
CE	4.64 ± 0.17	81.04 ± 0.18
CLIP	3.15 ± 0.09	81.08 ± 0.24
DINO	3.55 ± 0.25	81.10 ± 0.10
β-VAE	3.73 ± 0.25	80.96 ± 0.06

predictive distributions. Such post-hoc and data augmentation approaches are complementary to training-time remedies and can be readily applied in conjunction with them.

Training-time approaches such as label smoothing address calibration by directly modifying supervision. Subsequent work has extended this idea beyond the uniform redistribution proposed by Szegedy et al. [21]. Online label smoothing [25] dynamically constructs soft targets during training using model statistics, allowing the smoothing distribution to adapt over time rather than remaining fixed. Maher and Kull [14] propose instance-based label smoothing, where a teacher model is used to assign instance-dependent smoothing factors and non-uniformly distribute mass across classes based on their similarity to the true class. More recent methods incorporate class structure when designing smoothing distributions. In particular, Liu and JaJa [13] leverage autoencoder latent spaces and word embeddings to define class-based soft targets. Yang et al. [23] further connect latent variable modeling with adaptive supervision, showing that uncertainty in latent representations can naturally induce soft labeling behavior. These works highlight the importance of representation geometry when redistributing label mass and move beyond purely uniform or prediction-based smoothing strategies.

Despite these advances, existing approaches primarily focus on how smoothing mass should be distributed, not on *where* it should be distributed. To the best of our knowledge, prior work does not explicitly investigate sparsity as a design principle in label smoothing.

6 Limitations

While effective sparsity values typically fall in a small range ($k \in [1, 10]$), the optimal k can vary across architectures and datasets; selected values are reported in Appendix A. Thus, choosing k remains a practical consideration requiring tuning. Our class-level neighborhoods make SSA-LS efficient with only one dataset-level precomputation, but may miss instance-specific ambiguity, where different examples from the same class have different plausible alternatives.

7 Conclusion

We presented similarity-aware sparsity as a simple but powerful design principle for label smoothing. Our results show that calibration is governed not only by the smoothing strength, but also by the support on which smoothing mass is placed. SSA-LS operationalizes this principle with a drop-in target construction rule that concentrates probability mass on semantically plausible alternatives, consistently improving calibration across architectures and datasets while preserving accuracy. Through sparsity ablations, corrupted and random neighborhoods, latent-space comparisons, and frozen-feature experiments, we show that these gains arise from structured semantic support rather than sparsity alone. We further show that strong representations do not by themselves guarantee calibrated predictions; calibration improves when their semantic structure is explicitly reflected in the training targets. This highlights an important distinction: useful representations can reveal class structure, but the training objective must still use that structure appropriately. Overall, SSA-LS reframes label smoothing as a question of *where* uncertainty should be assigned, not merely how much uncertainty to add. These findings suggest that effective label smoothing should not simply soften labels, but should sparsely place the softened mass on the right alternatives.

References

[1] Siran Dai, Qianqian Xu, Peisong Wen, Yang Liu, and Qingming Huang. Exploring structural degradation in dense representations for self-supervised learning, 2025. URL https://arxiv.org/abs/2510.17299.

[2] Morris H. DeGroot and Stephen E. Fienberg. The comparison and evaluation of forecasters. In *Proceedings of the 1982 I.O.S. Annual Conference on Practical Bayesian Statistics*, pages 12–22, 1983.

[3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

[4] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, pages 2–3, 2017.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. URL http://arxiv.org/abs/1512.03385.

[6] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations*, 2017.

[7] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for mobilenetv3, 2019. URL https://arxiv.org/abs/1905.02244.

[8] Gao Huang, Zhuang Liu, and Kilian Q. Weinberger. Densely connected convolutional networks. *CoRR*, abs/1608.06993, 2016. URL http://arxiv.org/abs/1608.06993.

[9] Weiran Huang, Mingyang Yi, Xuyang Zhao, and Zihao Jiang. Towards the generalization of contrastive self-supervised learning, 2023. URL https://arxiv.org/abs/2111.00743.

[10] Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022.

[11] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. 2009.

[12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. 2012.

[13] Chihuang Liu and Joseph JaJa. Class-similarity based label smoothing for confidence calibration, 2021. URL https://arxiv.org/abs/2006.14028.

[14] Mohamed Maher and Meelis Kull. Instance-based label smoothing for better calibrated classification networks, 2021. URL https://arxiv.org/abs/2110.05355.

[15] Rafael Müller, Simon Kornblith, and Geoffrey E. Hinton. When does label smoothing help? *CoRR*, abs/1906.02629, 2019. URL http://arxiv.org/abs/1906.02629.

[16] Mahdi Pakdaman Naeini, Gregory F Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, page 2901, 2015.

[17] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22th International Conference on Machine Learning (ICML 2005)*, pages 625–632, 2005.

[18] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2024. URL https://arxiv.org/abs/2304.07193.

[19] Gabriel Pereyra, George Tucker, Jan Chorowski, Lukasz Kaiser, and Geoffrey E. Hinton. Regularizing neural networks by penalizing confident output distributions. *CoRR*, abs/1701.06548, 2017. URL http://arxiv.org/abs/1701.06548.

[20] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021. URL https://arxiv.org/abs/2103.00020.

[21] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015. URL http://arxiv.org/abs/1512.00567.

[22] Sunil Thulasidasan, Gopinath Chennupati, Jeff Bilmes, Tanmoy Bhattacharya, and Sarah E. Michalak. On mixup training: Improved calibration and predictive uncertainty for deep neural networks. 2019. doi: 10.2172/1525811. URL https://doi.org/10.2172/1525811.

[23] Sin-Han Yang, Zhedong Liu, Gian Maria Marconi, and Mohammad Emtiyaz Khan. Variational learning induces adaptive label smoothing, 2025. URL https://arxiv.org/abs/2502.07273.

[24] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks, 2017. URL https://arxiv.org/abs/1605.07146.

[25] Chang-Bin Zhang, Peng-Tao Jiang, Qibin Hou, Yunchao Wei, Qi Han, Zhen Li, and Ming-Ming Cheng. Delving deep into label smoothing. *IEEE Transactions on Image Processing*, 30: 5984–5996, 2021. ISSN 1941-0042. doi: 10.1109/tip.2021.3089942. URL http://dx.doi.org/10.1109/TIP.2021.3089942.

[26] Hanlin Zhang, Xuechen Li, Prithviraj Sen, Salim Roukos, and Tatsunori Hashimoto. A closer look at the calibration of differentially private learners, 2022. URL https://arxiv.org/abs/2210.08248.

A Training details

Classification models. All models are trained from scratch for 200 epochs using SGD with Nesterov momentum 0.9 and weight decay 5×10^{-4}. We use cosine learning-rate decay with an initial learning rate of 0.1 and batch size 128. For ResNet-18 and ResNet-34, we replace the ImageNet stem with a 3×3 convolution with stride 1 and remove the initial max-pooling layer; all other architectures use their standard small-input implementations with dataset-specific classifiers.

The MLP probe used in subsection 4.4 is trained with AdamW using a learning rate of 1×10^{-3}. All other evaluation settings are unchanged.

In-domain β-VAE. We train a β-VAE on CIFAR-100 and TinyImageNet as an additional latent space, where β is annealed linearly from 0 to 4 over the first 20% epochs. For the in-domain latent representation, we train a convolutional β-VAE separately on each dataset and use the latent mean μ to compute class centroids. The complete list of hyperparameters used is listed in Table 5.

Table 5: Training details for the in-domain β-VAE representations used to construct class neighborhoods.

Setting	CIFAR-100	TinyImageNet
Input resolution	32×32	64×64
Encoder/decoder depth	3 stages	3 stages
Normalization	GroupNorm	GroupNorm
Latent dimension	128	32
Batch size	256	256
Optimizer	Adam	Adam
Learning rate	2×10^{-4}	2×10^{-4}
Weight decay	2×10^{-5}	1×10^{-5}
Training epochs	300	300

Smoothing methods. For SSA-LS on CIFAR-100, the sparsity factor is set to $k = 10$ for DenseNet-121, $k = 10$ for MobileNetV3-Large, $k = 1$ for ResNet-18, $k = 5$ for ResNet-34, and $k = 1$ for

WideResNet-28-10. For TinyImageNet, the sparsity factor is set to $k = 15$ for DenseNet-121 and $k = 5$ for ResNet-18. Class neighborhoods are computed once per dataset and reused across seeds and architectures.

We adapt the official released code for the evaluated baselines: Online Label Smoothing[1] (OLS) and Instance-based Label Smoothing[2] (ILS). For OLS, we set $\alpha = 0.5$, as suggested by the authors.

Compute resources. Experiments were run on cloud GPU instances using NVIDIA RTX PRO 6000 GPUs. A single 200-epoch run required about 10-30 minutes, depending on architecture. Our method incurs no training-time computational overhead. The only additional overhead is the latent feature extraction and class-neighborhood precomputation, performed once per dataset and reused across seeds, architectures, and smoothing configurations. This precomputation took about 5 minutes for CIFAR-100 and 15 minutes for TinyImageNet. The full project also included additional preliminary sweeps over smoothing strength, sparsity k, and latent-space choices beyond the final reported experiments.

B Additional results on CIFAR-100 & TinyImageNet

We provide additional results on CIFAR-100 (Table 6) and TinyImageNet (Table 7) to support the main findings. These experiments support the main conclusion that the calibration benefits of SSA-LS are not specific to a single model or dataset. In all settings, SSA-LS reduces ECE relative to standard label smoothing and other soft-target baselines, while preserving or even improving top-1 accuracy.

Table 6: Calibration and accuracy comparison across additional architectures on CIFAR-100. We report results using Cross Entropy (CE), Uniform Label Smoothing (LS) [21], Online Label Smoothing (OLS) [25], Instance-based Label Smoothing (ILS) [14] and SSA-LS (ours) targets. Across all architectures, SSA-LS consistently achieves the lowest ECE, while MCE and NLL are also improved in most cases. Accuracy remains comparable to, or even exceeds, baseline methods.

Model	Method	ECE ↓	Accuracy ↑	NLL ↓	MCE ↓
MNv3-Large	CE	4.27 ± 0.27	56.05 ± 1.31	1.58 ± 0.05	25.06 ± 38.22
	LS	2.17 ± 0.18	56.08 ± 1.19	1.58 ± 0.04	6.21 ± 0.41
	OLS	12.41 ± 0.67	56.91 ± 1.41	1.64 ± 0.05	20.48 ± 0.81
	ILS	3.94 ± 0.38	55.75 ± 1.22	1.60 ± 0.06	7.69 ± 1.38
	SSA-LS	$\mathbf{2.01 \pm 0.21}$	$\mathbf{56.95 \pm 0.44}$	$\mathbf{1.54 \pm 0.02}$	$\mathbf{5.55 \pm 0.96}$
ResNet-34	CE	7.13 ± 0.44	79.30 ± 0.41	0.86 ± 0.01	20.32 ± 2.68
	LS	5.26 ± 0.51	79.46 ± 0.49	0.89 ± 0.02	$\mathbf{17.16 \pm 1.49}$
	OLS	9.78 ± 0.23	79.01 ± 0.16	0.89 ± 0.00	43.26 ± 28.52
	ILS	6.52 ± 0.57	79.16 ± 0.48	0.86 ± 0.01	19.22 ± 2.56
	SSA-LS	$\mathbf{4.83 \pm 0.80}$	$\mathbf{79.50 \pm 0.63}$	$\mathbf{0.81 \pm 0.02}$	17.25 ± 2.68

C Sparsity ablation results

We report the full numerical results corresponding to the sparsity analysis in subsection 4.3 in Table 8. Varying the neighborhood size k changes the support of the smoothing distribution, allowing us to quantify how sparsity affects calibration while monitoring its effect on classification accuracy.

D Sparsifying existing label smoothing methods

We assess whether similarity-aware sparsity can act as a general wrapper for existing label smoothing methods, and more importantly, when such a constraint is beneficial. We construct sparse variants of OLS [25] and ILS [14] by preserving each method's original target-generation mechanism and

[1] https://github.com/ankandrew/online-label-smoothing-pt
[2] https://github.com/mmaher22/Instance-based-smoothing

Table 7: Calibration and accuracy comparison across architectures on TinyImageNet. We report results using Cross Entropy (CE), Uniform Label Smoothing (LS) [21], Online Label Smoothing (OLS) [25], Instance-based Label Smoothing (ILS) [14] and SSA-LS (ours) targets. Across all architectures, SSA-LS consistently achieves the lowest ECE, while MCE and NLL are also improved in most cases. Accuracy remains comparable to, or even exceeds, baseline methods.

Model	Method	ECE ↓	Accuracy ↑	NLL ↓	MCE ↓
DenseNet-121	CE	14.49 ± 0.27	59.55 ± 0.16	1.90 ± 0.02	29.79 ± 1.40
	LS	7.57 ± 0.19	59.48 ± 0.33	1.84 ± 0.01	15.27 ± 0.58
	OLS	12.96 ± 0.50	$\mathbf{60.41 \pm 0.38}$	1.84 ± 0.02	25.72 ± 2.19
	ILS	14.08 ± 0.35	59.99 ± 0.29	1.88 ± 0.00	26.48 ± 1.10
	SSA-LS	$\mathbf{7.39 \pm 0.27}$	59.77 ± 0.33	$\mathbf{1.78 \pm 0.02}$	$\mathbf{14.92 \pm 1.62}$
ResNet-18	CE	7.26 ± 0.20	66.22 ± 0.22	1.52 ± 0.01	17.24 ± 1.45
	LS	6.02 ± 0.56	65.91 ± 0.62	1.59 ± 0.01	13.07 ± 1.60
	OLS	6.70 ± 0.15	$\mathbf{66.64 \pm 0.32}$	1.52 ± 0.00	16.37 ± 1.05
	ILS	7.13 ± 0.09	66.21 ± 0.16	1.51 ± 0.01	18.71 ± 2.17
	SSA-LS	$\mathbf{3.76 \pm 0.40}$	65.71 ± 0.14	$\mathbf{1.49 \pm 0.01}$	$\mathbf{8.85 \pm 1.58}$

Table 8: Effect of sparsity on calibration and performance. Varying the neighborhood size k shows that calibration is strongly affected by the support of the smoothing distribution, while accuracy remains largely stable. Small semantic neighborhoods yield the best ECE and MCE on both CIFAR-100 and TinyImageNet, indicating that smoothing mass should be concentrated on a limited set of plausible alternatives rather than spread broadly across many classes.

(a) CIFAR-100

Method	Sparsity (k)	ECE ↓	Accuracy ↑	NLL ↓	MCE ↓
CE	–	4.49 ± 0.16	78.45 ± 0.23	0.87 ± 0.01	14.58 ± 5.41
LS	–	5.84 ± 0.31	$\mathbf{79.13 \pm 0.10}$	0.93 ± 0.01	20.60 ± 1.56
SSA-LS	1	$\mathbf{2.34 \pm 0.35}$	78.62 ± 0.21	0.84 ± 0.01	$\mathbf{12.82 \pm 1.55}$
SSA-LS	3	3.06 ± 0.25	78.86 ± 0.24	$\mathbf{0.83 \pm 0.00}$	14.85 ± 1.79
SSA-LS	5	3.55 ± 0.22	79.02 ± 0.23	0.83 ± 0.01	15.15 ± 1.28
SSA-LS	10	4.39 ± 0.26	79.02 ± 0.35	0.85 ± 0.01	16.19 ± 0.83

(b) TinyImageNet

Method	Sparsity (k)	ECE ↓	Accuracy ↑	NLL ↓	MCE ↓
CE	–	7.25 ± 0.21	$\mathbf{66.22 \pm 0.22}$	1.52 ± 0.01	17.25 ± 1.39
LS	–	6.01 ± 0.56	65.91 ± 0.62	1.59 ± 0.01	13.12 ± 1.67
SSA-LS	1	4.37 ± 0.41	65.55 ± 0.26	1.50 ± 0.01	10.40 ± 1.04
SSA-LS	3	3.81 ± 0.26	65.38 ± 0.25	1.50 ± 0.01	9.14 ± 0.71
SSA-LS	5	$\mathbf{3.75 \pm 0.39}$	65.71 ± 0.14	$\mathbf{1.49 \pm 0.01}$	$\mathbf{8.82 \pm 1.65}$
SSA-LS	10	4.53 ± 0.25	65.84 ± 0.28	$\mathbf{1.49 \pm 0.01}$	9.21 ± 0.91

Table 9: Effect of similarity-aware sparsification on existing label smoothing methods. We construct sparse variants (SSA) of OLS [25] and ILS [14] that preserve each method's target-generation mechanism, as in Eq. 12. SSA-OLS improves both calibration and accuracy, suggesting that semantic support constraints sharpen dense class-level adaptive targets. SSA-ILS remains essentially unchanged, indicating that ILS is not primarily bottlenecked by support density.

(a) ResNet-34

Method	ECE ↓	Accuracy ↑
CE	7.13 ± 0.44	79.30 ± 0.41
LS	5.26 ± 0.51	79.46 ± 0.49
OLS	9.78 ± 0.23	79.01 ± 0.16
SSA-OLS	7.93 ± 0.98	79.54 ± 0.28
ILS	6.52 ± 0.57	79.16 ± 0.48
SSA-ILS	6.65 ± 0.21	79.12 ± 0.23

(b) DenseNet-121

Method	ECE ↓	Accuracy ↑
CE	10.26 ± 0.33	67.99 ± 0.41
LS	5.89 ± 0.53	67.64 ± 0.16
OLS	17.04 ± 0.35	66.93 ± 0.11
SSA-OLS	12.39 ± 0.44	67.60 ± 0.41
ILS	9.64 ± 0.39	67.81 ± 0.33
SSA-ILS	9.61 ± 0.44	68.08 ± 0.41

applying SSA only to the final target distribution: the target-class probability is unchanged, while the non-target mass is redistributed over the top-k semantic neighbors as in Eq. 12.

Table 9 shows a clear contrast between OLS and ILS. OLS already produces adaptive, non-uniform targets by accumulating prediction statistics for each class. Thus, SSA-OLS is not improving a uniform baseline; rather, it tests whether learned class-level soft targets benefit from an explicit semantic support constraint. The answer is yes: sparsifying OLS improves both calibration and accuracy across architectures. On ResNet-34, ECE decreases from 9.78 to 7.93 while accuracy increases from 79.01% to 79.54%; on DenseNet-121, ECE decreases from 17.04 to 12.39 while accuracy increases from 66.93% to 67.60%. These gains suggest that OLS learns useful class-level semantic structure, but that its learned targets still benefit from sharpening the non-target support.

In contrast, sparsifying ILS produces essentially no change in either calibration or accuracy. This is not simply due to applying a class-level neighborhood to an instance-based method: we also tested an instance-dependent sparsification variant and observed the same lack of improvement. This suggests that ILS is not primarily bottlenecked by dense non-target support. Instead, under our setting, its performance appears limited by other aspects of the teacher-driven target construction, such as teacher quality, smoothing magnitude estimation, or noise in instance-level probability estimates. Thus, while SSA substantially improves OLS by sharpening dense learned class-level targets, support restriction alone does not improve ILS.

These results clarify the role of similarity-aware sparsity. SSA does not merely make targets sparse; it supplies a missing support prior. This prior is beneficial when the base method has useful semantic structure but leaves non-target mass spread too broadly, as in OLS. When sparsification fails even at the instance level, as in ILS, the limiting factor likely lies elsewhere in the target construction. Therefore, SSA should be viewed not as a universal improvement to all soft targets, but as a targeted correction for methods whose non-target probability mass remains overly dense.